

Kapitel 1: Einführung

§1 Algorithmen | §2 Datenstrukturen | §3 Grundkonzepte

1 Algorithmus

Definition 1.1 — Algorithmus

Ein **Algorithmus** ist eine Problemlösung durch *endlich viele elementare Schritte*.

Problemspezifikation = Vorbedingung + Nachbedingung:

- **Vorbedingung (Precondition):** Beschreibt die erlaubten Eingaben.
- **Nachbedingung (Postcondition):** Beschreibt die geforderte Ausgabe.

Elementarer Schritt = Teilproblem, für das ein Algorithmus bereits bekannt ist (Elementare Operation).

Endliche Schritte = der Algorithmus terminiert immer (endliche Laufzeit).

Achtung

Laxe vs. rigorose Definition: In der Praxis wird oft die Nachbedingung durch den Algorithmus selbst definiert. In diesem Kurs gilt: Probleme werden *unabhängig* vom Algorithmus rigoros spezifiziert.

Beispiel 1.2 — Gleichseitiges Dreieck (Elementare Geometrie)

Elementare Operationen: Punkt markieren, Gerade durch 2 Punkte, Kreis um Punkt mit Radius, Abstand messen.

Konstruktion: Gegeben Punkte A, B .

1. Zeichne Kreis um A mit Radius $|AB|$.
2. Zeichne Kreis um B mit Radius $|AB|$.
3. Schnittpunkt C der beiden Kreise = dritter Eckpunkt.

Ergebnis: $\triangle ABC$ ist gleichseitig. $\Rightarrow$ Die lösbaren Probleme hängen direkt von der Operationsmenge ab!

2 Berechenbarkeitstheorie

Definition 1.3 — WHILE-Programm

Ein **WHILE-Programm** ist ein universelles Berechnungsmodell (äquivalent zu Turing-Maschine, λ -Kalkül, μ -Rekursion). Speicherzellen enthalten ganze Zahlen (Anfangswert: 0).

Operationen:

- $x[i] = x[i] + c$ (Addition einer Konstanten c)
- $x[i] = x[i] - c$ (Subtraktion einer Konstanten c)
- $P; P$ (Sequenzielle Komposition)
- **WHILE** $x[i] \neq 0$ **DO** P **DONE** (Schleife)

Modell	Erfinder	Bemerkung
Turing-Maschine	Alan Turing	Grundlage der Informatik
λ -Kalkül	Alonzo Church	funktionale Sprachen
μ -Rekursion	Kurt Gödel	Mathematik
Markov-Algorithmen	Andrey Markov	Zeichenkettenersetzung
WHILE-Programm	Plotkin, Nielson, Hankin	wird in dieser VL verwendet

In der Praxis: Python als höhere Programmiersprache mit erweiterter Operationsmenge.

3 Datenstruktur, Datentyp, ADT

Definition 1.4 — Datenstruktur / Datentyp / ADT

Datenstruktur: Ordnet eine Bitfolge in Gruppen und gibt jeder Gruppe eine *Bedeutung*.

Datentyp (Klasse): Speicherlayout + Operationen.

Abstrakter Datentyp (ADT): Nur die Menge der ausführbaren Operationen — *kein* Speicherlayout, *keine* Implementierung.

Beispiel 1.5 — Dieselbe Bitfolge, verschiedene Interpretationen

Bitfolge: 1101011001101100

Typ	Interpretation
uint16	54892
int16	−10644 (Zweierkomplement)
char[2]	"ö1" (Windows-Zeichensatz)
float16	−102.75 (IEEE 754)

⇒ Interpretation hängt vollständig von der gewählten Datenstruktur ab.

Begriff	Inhalt	Beispiel
Datentyp	Layout + Operationen	int32, float64
ADT	nur Operationen	Zahlen (passt für int, double, ...)
Explizite Typisierung	int a; double b	C, Pascal
Implizite Typisierung	a=5, a=5.0	Python
Statisch typisiert	Prüfung bei Compilierung	C, Java
Dynamisch typisiert	Prüfung bei Ausführung	Python

4 Wichtige Objekt-Begriffe

Definition 1.6 — Objekt, Identität, Zustand

- **Objekt / Instanz:** Speicherbereich, der den Anforderungen einer Klasse genügt.
- **Identität / Schlüssel:** Eindeutige Kennzeichnung eines Objekts (z. B. Speicheradresse).
- **Zustand / Wert:** Das Bitmuster bzw. dessen Interpretation.
- **Wertebereich:** Menge aller legalen Werte einer Klasse.
- **Referenz:** Objekt, das den Schlüssel eines anderen Objekts enthält (Zeiger / Pointer).

5 Fundamentale Algorithmen: Konstruktoren, Destruktoren, Vergleiche, Zuweisungen

Definition 1.7 — Konstruktor

Konstruktor: Erzeugt ein Objekt vom Typ T .

- Standard: $t = T()$ Vor: $T \in \text{Types}$, Nach: $t = T()$
- Mit Argument: $i = \text{int}(2)$, $f = \text{float}(1.5)$

Destruktor: Gibt Speicher frei. In Python automatisch durch Garbage Collection.

Definition 1.8 — Identity vs. Equality

- **Identity (Referenzgleichheit):** `is`, `is not` — sind es *dieselben* Objekte?
- **Equality (Wertgleichheit):** `==`, `!=` — haben sie *denselben Wert*?

Achtung

Identity $\Rightarrow$ Equality, aber nicht umgekehrt!

`a = 10; b = 10  $\Rightarrow$  a == b` ist `True`, aber `a is b` kann `False` sein.

(Python cached kleine Integers intern, aber das ist Implementierungsdetail — nicht verlassen!)

Definition 1.9 — Referenzzuweisung vs. Wertzuweisung

Referenzzuweisung (Standard in Python): `x = y`

- x und y zeigen auf *dasselbe* Objekt: `assert x is y`
- Vor: $y \in T$; Nach: $x \text{ is } y$

Wertzuweisung (Kopie): `x = copy.deepcopy(y)` oder `x = a.copy()` (flach)

- x ist ein *neues* Objekt mit gleichem Wert: `x == y` und `x is not y`

Beispiel 1.10 — Referenz- vs. Wertzuweisung in Python

```
import copy

# Referenzzuweisung
a = [1, 2, 3]
b = a          # b is a == True
b.append(4)
print(a)       # [1, 2, 3, 4] <- a wurde mitgeändert!

# Flache Kopie
```

```

c = a.copy()      # c is not a
c.append(5)
print(a)          # [1, 2, 3, 4]  <- a unverändert

# Tiefe Kopie
d = copy.deepcopy(a)  # vollstaendig unabhaengig

```

6 Mutability in Python

Definition 1.11 — Immutable vs. Mutable

Immutable: Wert kann nach Erstellung *nicht* geändert werden.

Beispiele: `int`, `float`, `str`, `tuple`

Mutable: Wert kann *in-place* geändert werden.

Beispiele: `list`, `dict`, `set`

Typ	Zuweisung <code>b = a</code>	Nach <code>b += 1</code> / <code>b.append(x)</code>
<code>int</code> (immutable)	neue Bindung	<code>a is not b</code> (neues Objekt)
<code>list</code> (mutable)	gleiche Referenz	<code>a is b</code> , <code>a</code> wird mitverändert!

Achtung

Sonderfall — Tuple von Arrays:

`a = ([1,2,3], [4,5])` — das Tuple ist immutable (keine neuen Elemente hinzufügar), aber `a[0][0] = 99` funktioniert, da die enthaltene Liste mutable ist!

⇒ Immutability des Containers schützt *nicht* die darin enthaltenen mutablen Objekte.

Beispiel 1.12 — Immutable vs. Mutable Verhalten

```

# Immutable: int
a = 10
b = a
b += 1
print(a, b)      # 10 11  (a unverändert, b ist neues Objekt)
print(a is b)    # False

# Mutable: list
x = [1, 2, 3]
y = x
y.append(4)
print(x)         # [1, 2, 3, 4]  (x wurde mitverändert!)
print(x is y)    # True

```

18 Themen des Kurses

1. Einführung: Algorithmen, Datenstrukturen *(dieses Kapitel)*
2. Asymptotische Komplexität
3. Elementare Container: Array, List, Stack, Queue
4. Effizienz: Schneller Code
5. Testing: Korrekter Code
6. Sortieren und Auswählen: Insertion-/Selection-/Merge-/Quicksort, Quartile
7. Sortieren in linearer Zeit: Counting-/Radix-/Bucketsort
8. Such-Algorithmen: Lineare/Binäre Suche, Suchbäume
9. Prioritätswarteschlangen: Heap, Heapsort, Bucket-Prinzip
10. Hashing: Hashtabellen (geschlossen/offen), Textsuche
11. Graphen: Definitionen und Darstellung
12. Graphdurchlauf: Breiten-/Tiefensuche, Toposort
13. Kürzeste Wege in Graphen
14. Minimaler Spannbaum: Kruskal und Prim
15. Greedy-Algorithmen und Dynamisches Programmieren
16. Rekursion vs. Iteration
17. P vs. NP: Schwierigkeitsklassen
18. Generisches Programmieren

Prüfungsscheckliste — Kapitel 1: Einführung

- ☐ Definition **Algorithmus** angeben: Was sind Vor-/Nachbedingung und elementare Schritte? → *Def. 1.1*
- ☐ **Problemspezifikation** erklären: Vorbedingung + Nachbedingung, Beispiel aufschreiben. → *Def. 1.1*
- ☐ **WHILE-Programm**: Die 4 Operationen nennen und erläutern. → *Def. 1.3*
- ☐ Andere Berechnungsmodelle nennen (Turing, Church, Gödel, Markov) und Äquivalenz erklären. → *§2*
- ☐ **Datenstruktur** vs. **Datentyp** vs. **ADT** klar unterscheiden. → *Def. 1.4*
- ☐ Beispiel für gleiche Bitfolge mit verschiedenen Typen vorführen. → *Bsp. 1.5*
- ☐ **Statisch** vs. **dynamisch** typisiert: Unterschied, Zeitpunkt der Prüfung, Python-Einordnung. → *§3*
- ☐ **Identity** (`is`) vs. **Equality** (`==`): Unterschied und Implikationsrichtung. → *Def. 1.8*
- ☐ Implikation *Identity* $\Rightarrow$ *Equality* beweisen; Gegenbeispiel für Umkehrung angeben. → *Def. 1.8, Achtung*
- ☐ **Referenzzuweisung** vs. **Wertzuweisung**: Python-Code, Vor-/Nachbedingung. → *Def. 1.9*
- ☐ **Mutable** vs. **Immutable**: Typbeispiele, Verhalten bei `b = a` und anschließender Mutation. → *Def. 1.11*
- ☐ Sonderfall **Tuple von Arrays**: Warum kann man `a[0][0]` ändern, obwohl Tuple immutable? → *Achtung, §6*
- ☐ **Konstruktor** spezifizieren: Standard und mit Argument, Vor-/Nachbedingung. → *Def. 1.7*
- ☐ Geometrie-Beispiel: Warum hängt die Lösbarkeit von der Operationsmenge ab? → *Bsp. 1.2*

